Why Different Approaches To Naming Classes

CSS is fairly unique in the world

- Separates content from appearance
- Applies appearance based on structure

Class names are important

- Better define your structure
    - And **communicate** that structure
- Like naming the rooms of your house
    - By purpose
    - Not numbered, not walls/floor/ceiling/door

Different Needs

Different pages/sites have different needs

- News/Informational sites
- Govt Agencies/Universities/Companies
- Different Product Areas for Sales/Support

Different breakdowns of **Audience**, **Context**, and **Goals**

CSS by Structure

You COULD style by type structure

- `div > ul > li > a`
- Hard to understand what you are styling
    - Is this a navigation? A recipe?
- Tweaking HTML breaks styling

What alternatives exist?

Semantic CSS

Give elements class names that describe the **purpose**

```
<div class="profile">
    <img class="avatar" src="avatar.png" alt="..."/>
    <span class="name">Jorts</span>
</div>
```

Using Semantic CSS

Approach: Use Cascading, use semantic class names

Pros:

- Original "intent" of CSS
- No conflicts between class names and visuals
- Keeps code maintainable

Cons:

- Have to consider names for many parts
- Uniquely naming purpose but not visuals is hard

Semantic CSS Example

- Style headings, lists, paragraphs
- Add specific classes for specific but common purposes (menus, callouts, etc)

```
<nav class="site-nav">
  <ul class="menu">
    <li><a href="about.html">About</a></li>
    <li><a href="famous.html">Famous Cats</a></li>
    <li><a href="lolcats.html">LOLCats</a></li>
  </ui>
</nav>
```

See also: **http://www.csszengarden.com/**

BEM

Not today:

- We will learn a naming style
- BEM (Block Element Modifier)

BEM is still semantic

- A pattern of how to name multiple related classes
- Reduces issues with similar names

But for now we will use semantic kebab-case names

- Make sure you know what I mean by **kebab-case**

Utility Classes

Semantic classes

- Named for what the element is/represents
- Not for the change the class imposes
    - Not for appearance

Utility classes

- Not named for what the element is/represents
- Named for what change the class imposes

Utility Class Examples

- `fade-in`
- `large`
- `fill-page-width`

These names usually describe some visual effect, but could name anything that is created by CSS.

The focus is on the effect created, not what the element itself *is* that makes it get that effect

Utility First / Atomic CSS

Approach: No semantics

- Focus on the added properties (Utility "first")
- Trying for 1 class = 1 CSS property

Pros:

- Once defined, easy to apply new content
- Less CSS in CSS file

Cons:

- MANY more class names in HTML
- Design changes = Many HTML changes

Similar to putting styles directly on an element

- Still allows Semantic HTML
    - But class names aren't semantic
- Some love it
    - Don't know CSS?
    - Know CSS but dislike it?
- Some hate it
    - Clutters up HTML
    - CSS class names have benefit

Utility First Examples

Many common libraries: Tailwind CSS, etc

```
<ul class="flex">
  <li class="mr-6">
    <a class="text-blue-500 hover:text-blue-800" href="#">Active</a>
  </li>
  <li class="mr-6">
    <a class="text-blue-500 hover:text-blue-800" href="#">Link</a>
  </li>
  <li class="mr-6">
    <a class="text-blue-500 hover:text-blue-800" href="#">Link</a>
  </li>
  <li class="mr-6">
    <a class="text-gray-400 cursor-not-allowed" href="#">Disabled</a>
  </li>
</ul>
```

Utility First is hard for this course

Because utility first

- Involves an external library
- Or a ton of work

Utility First is HAPPY to "avoid class names"

- But I'm TEACHING class names

Moving beyond this course

Start with Semantic CSS

- Easy to shift to BEM
    - Can do so in this course
- Easy to mix in a few utility classes!
- Utility First avoids the concepts we are teaching
    - Both the good and the bad
- You WILL encounter Semantic
    - Need to understand it
- In future:
    - All approaches valid depending on needs

Summary - Picking And Using Class Names

- Semantic
    - Classes named semantically
    - BEM (A fancy semantic)
- Utility Classes
    - Classes named for result of class
- Atomic/Utility First
    - 1 utility class = 1 applied effect
    - Classes named for that applied effect
    - MANY classes in HTML

Summary - This Course

- You **must use semantic class names**
  - They must be **kebab-case** or **BEM-style**
- You **may use BEM-style class names**
  - Not yet covered
- **We will not use utility classes**
- **We will not use atomic/utility first classes**